
PARABANK

Automation Framework

Complete Source Code

Prepared by

Suraj Shaw

Java SDET / QA Automation Project

June 2026

[GitHub Source Code Link](#)

Introduction

The ParaBank Automation Framework is a Java-based web automation project developed to validate the core banking workflows of the ParaBank demonstration application. The framework uses Selenium WebDriver and TestNG and follows the Page Object Model so that page locators, reusable actions, test data, execution logic, reporting, and configuration remain separated and maintainable.

Project Objective

The objective of this project is to create a reusable automation framework that executes positive and negative test scenarios, reads and writes test data through Excel, captures screenshots on failure, generates Extent Reports, supports TestNG suite execution, and can be integrated with Jenkins and Docker for CI/CD execution.

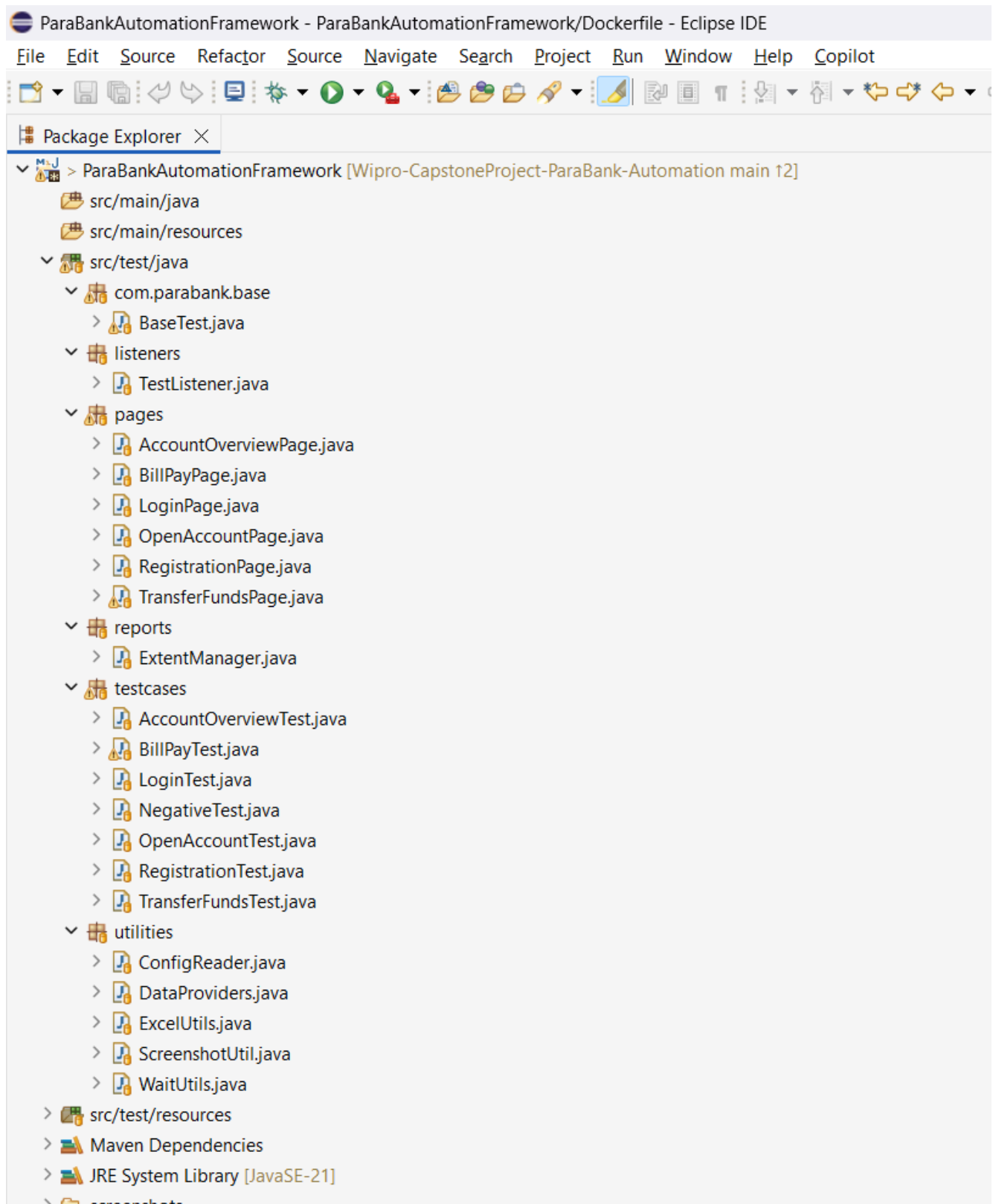
Automated Functional Modules

- Customer registration and generated-username storage
- Valid login using credentials read from Excel
- Account overview validation
- Opening a new bank account
- Transfer of funds between accounts with balance verification
- Bill payment using data-driven test inputs
- Invalid and empty-login negative validation

Tools and Technologies

Area	Technology
Programming language	Java
Automation	Selenium WebDriver
Test framework	TestNG
Build and dependency management	Maven
Design pattern	Page Object Model
Data-driven testing	Apache POI and Microsoft Excel
Reporting and logging	Extent Reports and Log4j2
Driver management	WebDriverManager
CI/CD and containerization	Jenkins and Docker

Framework Structure



- >  Maven Dependencies
- >  JRE System Library [JavaSE-21]
- >  screenshots
- >  src
-  target
- >  test-output
-  Dockerfile
-  > JenkinsFile
-  pom.xml
-  testng.xml
-  testng-negative.xml

Source Code Index

Category	Files	Purpose
Build and CI/CD	3	Dependencies, Docker image and Jenkins pipeline
Runtime configuration	4	TestNG, URL configuration and logging
Base framework	1	Browser setup and teardown
Page objects	6	Reusable locators and business actions
Utilities	5	Excel, configuration, waits and screenshots
Reporting and listener	2	Extent report lifecycle and TestNG events
Test classes	7	Functional and negative test scenarios

Complete Source Code

The following listings reproduce the source and configuration files from the uploaded Eclipse project. No artificial line-number column has been added.

Build, Containerization and CI/CD

pom.xml

```
pom.xml — pom.xml

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>ParaBankAutomationFramework</groupId>
  <artifactId>ParaBankAutomationFramework</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <dependencies>

    <!-- Selenium -->
    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>4.35.0</version>
    </dependency>

    <!-- TestNG -->
    <dependency>
      <groupId>org.testng</groupId>
      <artifactId>testng</artifactId>
      <version>7.11.0</version>
      <scope>test</scope>
    </dependency>

    <!-- WebDriverManager -->
    <dependency>
      <groupId>io.github.bonigarcia</groupId>
      <artifactId>webdrivermanager</artifactId>
      <version>6.1.0</version>
    </dependency>

    <!-- Apache POI -->
    <dependency>
      <groupId>org.apache.poi</groupId>
      <artifactId>poi-ooxml</artifactId>
      <version>5.4.1</version>
    </dependency>

  </dependencies>
</project>
```

pom.xml — pom.xml

```
<dependency>
  <groupId>com.aventstack</groupId>
  <artifactId>extentreports</artifactId>
  <version>5.1.2</version>
</dependency>
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-core</artifactId>
  <version>2.24.3</version>
</dependency>

<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-api</artifactId>
  <version>2.24.3</version>
</dependency>

<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-slf4j2-impl</artifactId>
  <version>2.24.3</version>
</dependency>
<dependency>
  <groupId>com.aventstack</groupId>
  <artifactId>extentreports</artifactId>
  <version>5.1.2</version>
</dependency>
</dependencies>
</project>
```

DockerFile

DockerFile — DockerFile

```
FROM maven:3.9.9-eclipse-temurin-21
WORKDIR /app
COPY pom.xml .
COPY testng.xml .
RUN mvn -B -DskipTests dependency:go-offline
COPY src ./src
CMD ["mvn", "-B", "clean", "test", "-Dheadless=true", "-Ddocker=true", "-Dselenium.grid.url=http://selenium-chrome:4444"]
```

JenkinsFile

JenkinsFile — JenkinsFile

```
pipeline {
  agent any
  options {
    skipDefaultCheckout(true)
    disableConcurrentBuilds()
    timestamps()
  }
  tools {
    maven 'Maven3'
    jdk 'jdk21'
  }
  environment {
    DOCKER_NETWORK      = 'parabank-network'
    SELENIUM_CONTAINER  = 'selenium-chrome'
    TEST_IMAGE          = "parabank-tests:${BUILD_NUMBER}"
    TEST_CONTAINER      = "parabank-tests-${BUILD_NUMBER}"
  }
  stages {
    stage('Checkout') {
      steps {
        deleteDir()
        checkout scm
      }
    }
    stage('Verify Tools') {
      steps {
        script {
          if (isUnix()) {
            sh 'java -version'
            sh 'git --version'
            sh 'docker version'
          } else {
            bat 'java -version'
            bat 'git --version'
            bat 'docker version'
          }
        }
      }
    }
    stage('Prepare Selenium Grid') {
      steps {
        script {
          if (isUnix()) {
            sh '''
              docker network inspect "$DOCKER_NETWORK" \
              >/dev/null 2>&1 || \
              docker network create "$DOCKER_NETWORK"
              docker rm -f "$SELENIUM_CONTAINER" \
              >/dev/null 2>&1 || true
            '''
          }
        }
      }
    }
  }
}
```

```

        docker run -d \
        --name "$SELENIUM_CONTAINER" \
        --network "$DOCKER_NETWORK" \
        -p 4444:4444 \
        --shm-size=2g \
        selenium/standalone-chrome:latest
    ...
    timeout(time: 3, unit: 'MINUTES') {
        waitUntil {
            int status = sh(
                returnStatus: true,
                script: '''
                    curl -fsS \
                    http://localhost:4444/status \
                    | grep -q '"ready":true'
                ...
            )
            if (status != 0) {
                echo 'Waiting for Selenium Grid...'
                sleep 3
            }
            return status == 0
        }
    }
    echo 'Selenium Grid is ready.'
} else {
    bat '''
        docker network inspect %DOCKER_NETWORK% >nul 2>&1 || docker network create %DOCKER_NETWORK%
        docker rm -f %SELENIUM_CONTAINER% >nul 2>&1
        docker run -d ^
        --name %SELENIUM_CONTAINER% ^
        --network %DOCKER_NETWORK% ^
        -p 4444:4444 ^
        --shm-size=2g ^
        selenium/standalone-chrome:latest
    ...
    powershell '''
        $ready = $false
        for ($i = 1; $i -le 60; $i++) {
            try {
                $response = Invoke-RestMethod `
                    -Uri "http://localhost:4444/status" `
                    -TimeoutSec 5
                if ($response.value.ready -eq $true) {
                    Write-Host "Selenium Grid is ready."
                    $ready = $true
                    break
                }
            }
            catch {
                Write-Host "Waiting for Selenium Grid..."
            }
        }
    }
}

```

```

        }
        Start-Sleep -Seconds 3
    }
    if (-not $ready) {
        Write-Host "Selenium Grid did not become ready."
        docker logs $env:SELENIUM_CONTAINER
        exit 1
    }
    ...
}
}
}
}
stage('Build Docker Image') {
    steps {
        dir('ParaBankAutomationFramework') {
            script {
                if (isUnix()) {
                    sh '''
                        docker build \
                        -f Dockerfile \
                        -t "$TEST_IMAGE" .
                    ...
                } else {
                    bat '''
                        docker build ^
                        -f Dockerfile ^
                        -t %TEST_IMAGE% .
                    ...
                }
            }
        }
    }
}
stage('Run Tests in Docker') {
    steps {
        dir('ParaBankAutomationFramework') {
            script {
                int testStatus
                if (isUnix()) {
                    sh '''
                        docker rm -f "$TEST_CONTAINER" \
                        >/dev/null 2>&1 || true
                    ...
                testStatus = sh(
                    returnStatus: true,
                    script: '''
                        docker run \
                        --name "$TEST_CONTAINER" \
                        --network "$DOCKER_NETWORK" \
                        "$TEST_IMAGE"
                    ...
                )
            }
        }
    }
}

```

```

    ...
)
sh '''
rm -rf target/surefire-reports
rm -rf test-output
rm -rf screenshots
mkdir -p target
docker cp \
"$TEST_CONTAINER":/app/target/surefire-reports \
target/ || true
docker cp \
"$TEST_CONTAINER":/app/test-output \
. || true
docker cp \
"$TEST_CONTAINER":/app/screenshots \
. || true
docker rm -f "$TEST_CONTAINER" \
>/dev/null 2>&1 || true
...
} else {
bat(
    returnStatus: true,
    script: '''
        docker rm -f %TEST_CONTAINER%
    ...
    )
testStatus = bat(
    returnStatus: true,
    script: '''
        docker run ^
        --name %TEST_CONTAINER% ^
        --network %DOCKER_NETWORK% ^
        %TEST_IMAGE%
    ...
    )
bat '''
    if exist target\\surefire-reports (
        rmdir /s /q target\\surefire-reports
    )
    if exist test-output (
        rmdir /s /q test-output
    )
    if exist screenshots (
        rmdir /s /q screenshots
    )
    if not exist target (
        mkdir target
    )
    ...
bat(
    returnStatus: true,

```

```

        script: '''
            docker cp ^
            %TEST_CONTAINER%:/app/target/surefire-reports ^
            target\\
        '''
    )
    bat(
        returnStatus: true,
        script: '''
            docker cp ^
            %TEST_CONTAINER%:/app/test-output ^
            .
        '''
    )
    bat(
        returnStatus: true,
        script: '''
            docker cp ^
            %TEST_CONTAINER%:/app/screenshots ^
            .
        '''
    )
    bat(
        returnStatus: true,
        script: '''
            docker rm -f %TEST_CONTAINER%
        '''
    )
}
if (testStatus != 0) {
    error('Tests failed inside Docker container.')
}
}
}
}
}
}
}
post {
    always {
        junit(
            testResults:
                'ParaBankAutomationFramework/target/surefire-reports/*.xml',
            allowEmptyResults: true
        )
        archiveArtifacts(
            artifacts:
                'ParaBankAutomationFramework/target/surefire-reports/**/*,' +
                'ParaBankAutomationFramework/test-output/**/*,' +
                'ParaBankAutomationFramework/screenshots/**/*',
            allowEmptyArchive: true
        )
    }
}

```

```
publishHTML(target: [
    allowMissing: true,
    alwaysLinkToLastBuild: true,
    keepAll: true,
    reportDir:
        'ParaBankAutomationFramework/test-output',
    reportFiles:
        'ExtentReport.html',
    reportName:
        'ParaBank Extent Report'
])
script {
    if (isUnix()) {
        sh '''
            docker rm -f "$TEST_CONTAINER" \
            >/dev/null 2>&1 || true
            docker rm -f "$SELENIUM_CONTAINER" \
            >/dev/null 2>&1 || true
            docker image rm -f "$TEST_IMAGE" \
            >/dev/null 2>&1 || true
            ...
        '''
    } else {
        bat(
            returnStatus: true,
            script: '''
                docker rm -f %TEST_CONTAINER%
                docker rm -f %SELENIUM_CONTAINER%
                docker image rm -f %TEST_IMAGE%
                ...
            '''
        )
    }
}
}
success {
    echo 'ParaBank Docker test execution completed successfully.'
}
failure {
    echo 'ParaBank Docker test execution failed. Check reports and screenshots.'
}
}
```

Test Suite and Runtime Configuration

testng.xml

testng.xml — testng.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">

<suite name="ParaBank Suite">

  <test name="ParaBank Tests">

    <classes>

      <class name="testcases.RegistrationTest"/>
      <class name="testcases.LoginTest"/>
      <class name="testcases.OpenAccountTest"/>
      <class name="testcases.TransferFundsTest"/>
      <class name="testcases.BillPayTest"/>
      <class name="testcases.NegativeTest"/>

    </classes>

  </test>
  <listeners>

    <listener
      class-name="listeners.TestListener"/>

  </listeners>

</suite>
```

testng-negative.xml

testng-negative.xml — testng-negative.xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">

<suite name="ParaBank Negative Suite" verbose="0">

  <listeners>
    <listener class-name="listeners.TestListener"/>
  </listeners>

</suite>
```

testng-negative.xml — testng-negative.xml

```
<test name="Negative Login Tests">
  <classes>
    <class name="testcases.NegativeTest"/>
  </classes>
</test>

</suite>
```

src/test/resources/config/config.properties

config.properties — src/test/resources/config/config.properties

```
url=https://parabank.parasoft.com/parabank/index.htm
browser=chrome
timeout=10
```

src/test/resources/log4j2.xml

log4j2.xml — src/test/resources/log4j2.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<Configuration status="OFF">

  <Appenders>

    <Console name="Console" target="SYSTEM_OUT">
      <PatternLayout pattern="%msg%n"/>
    </Console>

  </Appenders>

  <Loggers>

    <!-- Disable WebDriverManager Logs -->
    <Logger name="io.github.bonigarcia.wdm" level="OFF" additivity="false"/>

    <!-- Disable Selenium Logs -->
    <Logger name="org.openqa.selenium" level="OFF" additivity="false"/>

    <!-- Disable TestNG Logs -->
    <Logger name="org.testng" level="OFF" additivity="false"/>

    <!-- Root Logger -->
```

log4j2.xml — src/test/resources/log4j2.xml

```
<Root level="ERROR">
  <AppenderRef ref="Console"/>
</Root>

</Loggers>
</Configuration>
```

Base Framework

src/test/java/com/parabank/base/BaseTest.java

BaseTest.java — src/test/java/com/parabank/base/BaseTest.java

```
package com.parabank.base;

import java.net.MalformedURLException;
import java.net.URL;
import java.time.Duration;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.remote.RemoteWebDriver;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;

import io.github.bonigarcia.wdm.WebDriverManager;
import utilities.ConfigReader;

public class BaseTest {

    protected WebDriver driver;

    @BeforeMethod
    public void setup() {
        System.setProperty("webdriver.http.factory", "jdk-http-client");
        System.setProperty("wdm.quiet", "true");
        System.setProperty("webdriver.chrome.silentOutput", "true");

        java.util.logging.Logger.getLogger("org.openqa.selenium").setLevel(java.util.logging.Level.SEVERE);

        ChromeOptions options = new ChromeOptions();

        boolean headless = Boolean.parseBoolean(
            System.getProperty("headless", "false"));

        boolean docker = Boolean.parseBoolean(
            System.getProperty("docker", "false"));

        if (headless) {
            options.addArguments("--headless=new");
            options.addArguments("--no-sandbox");
            options.addArguments("--disable-dev-shm-usage");
            options.addArguments("--window-size=1920,1080");
        }

        if (docker) {
```

```
String gridUrl = System.getProperty(
    "selenium.grid.url",
    "http://selenium-chrome:4444");

try {
    driver = new RemoteWebDriver(
        new URL(gridUrl),
        options);

} catch (MalformedURLException e) {
    throw new RuntimeException(
        "Invalid Selenium Grid URL: " + gridUrl,
        e);
}

} else {

    WebDriverManager.chromedriver().setup();
    driver = new ChromeDriver(options);
}

if (!headless) {
    driver.manage().window().maximize();
}

driver.manage().timeouts()
    .implicitlyWait(Duration.ofSeconds(10));

driver.get(ConfigReader.getProperty("url"));
}

@AfterMethod
public void tearDown() {

    if (driver != null) {
        driver.quit();
    }
}

public WebDriver getDriver() {
return driver;
}
}
```

Page Object Classes

src/test/java/pages/RegistrationPage.java

RegistrationPage.java — src/test/java/pages/RegistrationPage.java

```
package pages;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public class RegistrationPage {

    WebDriver driver;
    WebDriverWait wait;

    public RegistrationPage(WebDriver driver) {
        this.driver = driver;
        wait = new WebDriverWait(driver, Duration.ofSeconds(20));
    }

    // Locators
    By registerLink = By.linkText("Register");

    By firstName = By.id("customer.firstName");
    By lastName = By.id("customer.lastName");
    By address = By.id("customer.address.street");
    By city = By.id("customer.address.city");
    By state = By.id("customer.address.state");
    By zipCode = By.id("customer.address.zipCode");
    By phoneNumber = By.id("customer.phoneNumber");
    By ssn = By.id("customer.ssn");

    By username = By.id("customer.username");
    By password = By.id("customer.password");
    By confirmPassword = By.id("repeatedPassword");

    By registerButton = By.xpath("//input[@value='Register']");

    public void registerUser(String fName, String lName, String addr, String cityName, String stateName, String zip,
        String phone, String userName, String passWord) {

        wait.until(ExpectedConditions.elementToBeClickable(registerLink)).click();

        wait.until(ExpectedConditions.visibilityOfElementLocated(firstName));

        driver.findElement(firstName).clear();
```

RegistrationPage.java — src/test/java/pages/RegistrationPage.java

```
        driver.findElement(firstName).sendKeys(fName);

        driver.findElement.lastName().clear();
        driver.findElement.lastName().sendKeys(lName);

        driver.findElement(address).clear();
        driver.findElement(address).sendKeys(addr);

        driver.findElement(city).clear();
        driver.findElement(city).sendKeys(cityName);

        driver.findElement(state).clear();
        driver.findElement(state).sendKeys(stateName);

        driver.findElement(zipCode).clear();
        driver.findElement(zipCode).sendKeys(zip);

        driver.findElement(phoneNumber).clear();
        driver.findElement(phoneNumber).sendKeys(phone);

        driver.findElement(ssn).clear();
        driver.findElement(ssn).sendKeys("123456");

        driver.findElement(username).clear();
        driver.findElement(username).sendKeys(userName);

        driver.findElement(password).clear();
        driver.findElement(password).sendKeys(passWord);

        driver.findElement(confirmPassword).clear();
        driver.findElement(confirmPassword).sendKeys(passWord);

        driver.findElement(registerButton).click();
    }
}
```

src/test/java/pages/LoginPage.java

LoginPage.java — src/test/java/pages/LoginPage.java

```
package pages;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.TimeoutException;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
```

```
import org.openqa.selenium.support.ui.WebDriverWait;

public class LoginPage {

    private WebDriver driver;

    private By usernameField = By.name("username");
    private By passwordField = By.name("password");
    private By loginButton =
        By.cssSelector("input[value='Log In']");

    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    public void login(String username, String password) {

        WebDriverWait wait =
            new WebDriverWait(driver, Duration.ofSeconds(15));

        wait.until(
            ExpectedConditions.visibilityOfElementLocated(
                usernameField))
            .sendKeys(username);

        driver.findElement(passwordField).sendKeys(password);

        driver.findElement(loginButton).click();
    }

    public boolean isLoginSuccessful() {
        try {
            WebDriverWait wait =
                new WebDriverWait(driver, Duration.ofSeconds(15));

            wait.until(
                ExpectedConditions.visibilityOfElementLocated(
                    By.xpath(
                        "//h1[normalize-space()='Accounts Overview']")));

            return true;
        } catch (TimeoutException e) {
            return false;
        }
    }

    public String getLoginErrorMessage() {
        try {
```

LoginPage.java — src/test/java/pages/LoginPage.java

```
        WebDriverWait wait =
            new WebDriverWait(driver, Duration.ofSeconds(5));

        return wait.until(
            ExpectedConditions.visibilityOfElementLocated(
                By.cssSelector("#rightPanel .error")))
            .getText()
            .trim();

    } catch (TimeoutException e) {
        return "No error message displayed";
    }
}
```

src/test/java/pages/AccountOverviewPage.java

AccountOverviewPage.java — src/test/java/pages/AccountOverviewPage.java

```
package pages;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public class AccountOverviewPage {

    WebDriver driver;
    WebDriverWait wait;

    public AccountOverviewPage(WebDriver driver) {

        this.driver = driver;

        wait = new WebDriverWait(driver, Duration.ofSeconds(20));
    }

    By accountsOverviewLink = By.linkText("Accounts Overview");

    By balance = By.xpath("(//table[@id='accountTable']//tr[1]/td[2])[1]");

    public double getCurrentBalance() {

        wait.until(ExpectedConditions.elementToBeClickable(accountsOverviewLink)).click();
    }
}
```

AccountOverviewPage.java — src/test/java/pages/AccountOverviewPage.java

```
String bal = wait.until(ExpectedConditions.visibilityOfElementLocated(balance)).getText();

bal = bal.replace("$", "").replace(",", "").trim();

return Double.parseDouble(bal);
}
}
```

src/test/java/pages/OpenAccountPage.java

OpenAccountPage.java — src/test/java/pages/OpenAccountPage.java

```
package pages;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.Select;
import org.openqa.selenium.support.ui.WebDriverWait;

public class OpenAccountPage {

    WebDriver driver;
    WebDriverWait wait;

    public OpenAccountPage(WebDriver driver) {
        this.driver = driver;
        wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    }

    By openNewAccountLink = By.linkText("Open New Account");
    By accountTypeDropdown = By.id("type");
    By fromAccountDropdown = By.id("fromAccountId");
    By openAccountButton = By.xpath("//input[@value='Open New Account']");
    By newAccountNumber = By.id("newAccountId");

    public String openSavingsAccount() {

        wait.until(ExpectedConditions.elementToBeClickable(openNewAccountLink)).click();

        wait.until(ExpectedConditions.visibilityOfElementLocated(accountTypeDropdown));

        Select accountType = new Select(driver.findElement(accountTypeDropdown));
        accountType.selectByVisibleText("SAVINGS");
    }
}
```

OpenAccountPage.java — src/test/java/pages/OpenAccountPage.java

```
wait.until(ExpectedConditions.visibilityOfElementLocated(fromAccountDropdown));

Select fromAccount = new Select(driver.findElement(fromAccountDropdown));
fromAccount.selectByIndex(0);

wait.until(ExpectedConditions.elementToBeClickable(openAccountButton)).click();

WebElement accountElement =
    wait.until(ExpectedConditions.visibilityOfElementLocated(newAccountNumber));

String accountNumber = accountElement.getText();

System.out.println("Account Number = " + accountNumber);

return accountNumber;
}
}
```

src/test/java/pages/TransferFundsPage.java

TransferFundsPage.java — src/test/java/pages/TransferFundsPage.java

```
package pages;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.Select;
import org.openqa.selenium.support.ui.WebDriverWait;

public class TransferFundsPage {

    WebDriver driver;
    WebDriverWait wait;

    public TransferFundsPage(WebDriver driver) {

        this.driver = driver;
        wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    }

    By transferFundsLink = By.linkText("Transfer Funds");

    By amount = By.id("amount");
```

```
By fromAccount = By.id("fromAccountId");
By toAccount = By.id("toAccountId");
By transferButton = By.xpath("//input[@value='Transfer']");
By successMessage = By.xpath("//h1");

public void validateTransferAmount(double currentBalance, double transferAmount) {
    if (transferAmount > currentBalance) {
        throw new RuntimeException("Insufficient Balance. Available Balance = " + currentBalance);
    }
}

public String[] transferFunds(String transferAmount) {
    wait.until(ExpectedConditions.elementToBeClickable(transferFundsLink)).click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(amount));
    driver.findElement(amount).clear();
    driver.findElement(amount).sendKeys(transferAmount);
    wait.until(ExpectedConditions.visibilityOfElementLocated(fromAccount));
    Select from = new Select(driver.findElement(fromAccount));
    Select to = new Select(driver.findElement(toAccount));
    from.selectByIndex(0);
    if (to.getOptions().size() > 1) {
        to.selectByIndex(1);
    } else {
        to.selectByIndex(0);
    }
    String fromAccNo = from.getFirstSelectedOption().getText();
    String toAccNo = to.getFirstSelectedOption().getText();
    System.out.println("From Account : " + fromAccNo);
    System.out.println("To Account : " + toAccNo);
    wait.until(ExpectedConditions.elementToBeClickable(transferButton)).click();
    wait.until(ExpectedConditions.visibilityOfElementLocated(successMessage));
}
```

TransferFundsPage.java — src/test/java/pages/TransferFundsPage.java

```
    return new String[] { fromAccNo, toAccNo };
}

public void transferFundsToSameAccount(String transferAmount) {
    wait.until(ExpectedConditions.elementToBeClickable(transferFundsLink)).click();

    wait.until(ExpectedConditions.visibilityOfElementLocated(amount));

    driver.findElement(amount).clear();
    driver.findElement(amount).sendKeys(transferAmount);

    Select from = new Select(driver.findElement(fromAccount));
    Select to = new Select(driver.findElement(toAccount));

    from.selectByIndex(0);
    to.selectByIndex(0);

    wait.until(ExpectedConditions.elementToBeClickable(transferButton)).click();
}
}
```

src/test/java/pages/BillPayPage.java

BillPayPage.java — src/test/java/pages/BillPayPage.java

```
package pages;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public class BillPayPage {
    WebDriver driver;
    WebDriverWait wait;

    public BillPayPage(WebDriver driver) {
        this.driver = driver;

        wait = new WebDriverWait(driver, Duration.ofSeconds(30));
    }
}
```

```
// Locators
By billPayLink = By.linkText("Bill Pay");

By payeeName = By.name("payee.name");

By address = By.name("payee.address.street");

By city = By.name("payee.address.city");

By state = By.name("payee.address.state");

By zipCode = By.name("payee.address.zipCode");

By phone = By.name("payee.phoneNumber");

By accountNumber = By.name("payee.accountNumber");

By verifyAccount = By.name("verifyAccount");

By amount = By.name("amount");

By sendPaymentButton = By.xpath("//input[@value='Send Payment']");

By successMessage = By.tagName("h1");

public void payBill(String payee, String street, String cityName, String stateName, String zip, String mobile,
    String accNo, String billAmount) {

    wait.until(ExpectedConditions.elementToBeClickable(billPayLink)).click();

    wait.until(ExpectedConditions.visibilityOfElementLocated(payeeName)).sendKeys(payee);

    driver.findElement(address).sendKeys(street);
    driver.findElement(city).sendKeys(cityName);
    driver.findElement(state).sendKeys(stateName);
    driver.findElement(zipCode).sendKeys(zip);
    driver.findElement(phone).sendKeys(mobile);

    driver.findElement(accountNumber).sendKeys(accNo);
    driver.findElement(verifyAccount).sendKeys(accNo);

    driver.findElement(amount).sendKeys(billAmount);

    wait.until(ExpectedConditions.elementToBeClickable(sendPaymentButton)).click();
}

public String getPaymentMessage() {

    return wait.until(ExpectedConditions.visibilityOfElementLocated(successMessage)).getText();
}
}
```

Utility Classes

src/test/java/utilities/ConfigReader.java

ConfigReader.java — src/test/java/utilities/ConfigReader.java

```
package utilities;

import java.io.InputStream;
import java.util.Properties;

public class ConfigReader {

    private static Properties prop;

    static {
        try {
            prop = new Properties();

            InputStream fis = ConfigReader.class.getClassLoader().getResourceAsStream("config/config.properties");

            prop.load(fis);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public static String getProperty(String key) {
        return prop.getProperty(key);
    }
}
```

DataProviders.java — src/test/java/utilities/DataProviders.java

```
package utilities;

import org.testng.annotations.DataProvider;

public class DataProviders {

    @DataProvider(name = "billPayData")
    public Object[][] getBillPayData() {

        return new Object[][] {

            {
                "Electricity Board",
                "Park Street",
                "Kolkata",
                "West Bengal",
                "700001",
                "9876543210",
                "12345",
                "100"
            },

            {
                "Water Department",
                "MG Road",
                "Kolkata",
                "West Bengal",
                "700002",
                "9999999999",
                "67890",
                "200"
            }
        };
    }
}
```

ExcelUtils.java — src/test/java/utilities/ExcelUtils.java

```
package utilities;

import java.io.FileInputStream;
import java.io.FileOutputStream;

import org.apache.poi.ss.usermodel.Cell;
import org.apache.poi.ss.usermodel.Row;
import org.apache.poi.xssf.usermodel.XSSFSheet;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;

public class ExcelUtils {

    private static final String FILE_PATH = "src/test/resources/testdata/ParabankData.xlsx";

    public static String getCellData(String sheetName, int rowNum, int colNum) {

        try {

            FileInputStream fis = new FileInputStream(FILE_PATH);

            XSSFWorkbook workbook = new XSSFWorkbook(fis);

            XSSFSheet sheet = workbook.getSheet(sheetName);

            if (sheet == null) {
                System.out.println("Sheet not found: " + sheetName);
                workbook.close();
                return "";
            }

            if (sheet.getRow(rowNum) == null) {
                System.out.println("Row not found: " + rowNum);
                workbook.close();
                return "";
            }

            if (sheet.getRow(rowNum).getCell(colNum) == null) {
                System.out.println("Cell not found");
                workbook.close();
                return "";
            }

            String value = sheet.getRow(rowNum).getCell(colNum).toString();

            workbook.close();

            return value;

        } catch (Exception e) {
```

```
        e.printStackTrace();
        return "";
    }
}

public static void setCellData(String sheetName, int rowNum, int colNum, String value) {
    try {
        FileInputStream fis = new FileInputStream(FILE_PATH);

        XSSFWorkbook workbook = new XSSFWorkbook(fis);

        XSSFSheet sheet = workbook.getSheet(sheetName);

        Row row = sheet.getRow(rowNum);

        if (row == null) {
            row = sheet.createRow(rowNum);
        }

        Cell cell = row.getCell(colNum);

        if (cell == null) {
            cell = row.createCell(colNum);
        }

        cell.setCellValue(value);

        FileOutputStream fos = new FileOutputStream(FILE_PATH);

        workbook.write(fos);

        fos.close();
        workbook.close();
        fis.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

src/test/java/utilities/ScreenshotUtil.java

ScreenshotUtil.java — src/test/java/utilities/ScreenshotUtil.java

```
package utilities;

import java.io.File;
import java.io.IOException;
import java.text.SimpleDateFormat;
import java.util.Date;

import org.apache.commons.io.FileUtils;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;

public class ScreenshotUtil {

    public static String captureScreenshot(WebDriver driver, String testName) throws IOException {

        String timestamp = new SimpleDateFormat("yyyyMMdd_HHmmss").format(new Date());

        String path = "screenshots/" + testName + "_" + timestamp + ".png";

        File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);

        FileUtils.copyFile(src, new File(path));

        return path;
    }
}
```

src/test/java/utilities/WaitUtils.java

WaitUtils.java — src/test/java/utilities/WaitUtils.java

```
package utilities;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public class WaitUtils {

    public static WebElement waitForElementVisible(WebDriver driver, By locator) {
```

```
        WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(20));  
        return wait.until(ExpectedConditions.visibilityOfElementLocated(locator));  
    }  
  
    public static WebElement waitForElementClickable(WebDriver driver, By locator) {  
        WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(20));  
        return wait.until(ExpectedConditions.elementToBeClickable(locator));  
    }  
}
```

Reporting and Listeners

src/test/java/reports/ExtentManager.java

ExtentManager.java — src/test/java/reports/ExtentManager.java

```
package reports;

import com.aventstack.extentreports.ExtentReports;
import com.aventstack.extentreports.reporter.ExtentSparkReporter;

public class ExtentManager {

    private static ExtentReports extent;

    public static ExtentReports getInstance() {

        if (extent == null) {

            ExtentSparkReporter spark = new ExtentSparkReporter("test-output/ExtentReport.html");

            spark.config().setDocumentTitle("ParaBank Automation Report");

            spark.config().setReportName("Execution Report");

            extent = new ExtentReports();
            extent.attachReporter(spark);

            extent.setSystemInfo("Tester", "Suraj Shaw");

        }

        return extent;

    }
}
```

src/test/java/listeners/TestListener.java

TestListener.java — src/test/java/listeners/TestListener.java

```
package listeners;

import java.io.IOException;

import org.testng.ITestListener;
import org.testng.ITestResult;

import com.aventstack.extentreports.ExtentReports;
import com.aventstack.extentreports.ExtentTest;
```

```
import com.parabank.base.BaseTest;

import reports.ExtentManager;
import utilities.ScreenshotUtil;

public class TestListener implements ITestListener {

    ExtentReports extent = ExtentManager.getInstance();

    ExtentTest test;

    @Override
    public void onTestStart(ITestResult result) {

        test = extent.createTest(result.getName());
    }

    @Override
    public void onTestSuccess(ITestResult result) {

        test.pass("Test Passed");

        System.out.println("✓ " + result.getName() + " PASSED");
    }

    @Override
    public void onTestFailure(ITestResult result) {

        test.fail(result.getThrowable());

        try {

            BaseTest base = (BaseTest) result.getInstance();

            String path = ScreenshotUtil.captureScreenshot(base.getDriver(), result.getName());

            test.addScreenCaptureFromPath("../" + path);

            System.out.println("📸 Screenshot Captured");

        } catch (IOException e) {

            e.printStackTrace();
        }

        System.out.println("✗ " + result.getName() + " FAILED");
    }

    @Override
    public void onFinish(org.testng.ITestContext context) {
```

TestListener.java — src/test/java/listeners/TestListener.java

```
    extent.flush();  
  }  
}
```

Test Classes

src/test/java/testcases/RegistrationTest.java

RegistrationTest.java — src/test/java/testcases/RegistrationTest.java

```
package testcases;  
  
import org.testng.Assert;  
import org.testng.annotations.Test;  
  
import com.parabank.base.BaseTest;  
  
import pages.RegistrationPage;  
import utilities.ExcelUtils;  
  
public class RegistrationTest extends BaseTest {  
  
    @Test  
    public void verifyRegistration() {  
  
        String firstName = ExcelUtils.getCellData("Registration", 1, 0);  
  
        String lastName = ExcelUtils.getCellData("Registration", 1, 1);  
  
        String address = ExcelUtils.getCellData("Registration", 1, 2);  
  
        String city = ExcelUtils.getCellData("Registration", 1, 3);  
  
        String state = ExcelUtils.getCellData("Registration", 1, 4);  
  
        String zipCode = ExcelUtils.getCellData("Registration", 1, 5);  
  
        String phone = ExcelUtils.getCellData("Registration", 1, 6);  
  
        String username = "suraj" + System.currentTimeMillis();  
  
        String password = "Test@123";  
  
        RegistrationPage registrationPage = new RegistrationPage(driver);  
  
        registrationPage.registerUser(firstName, lastName, address, city, state, zipCode, phone, username, password);  
    }  
}
```

RegistrationTest.java — src/test/java/testcases/RegistrationTest.java

```
Assert.assertTrue(driver.getPageSource().contains("Your account was created successfully"));

System.out.println("✓ Registration Successful");
ExcelUtils.setCellData("LoginData", 1, 0, username);

ExcelUtils.setCellData("LoginData", 1, 1, password);

System.out.println("Username Saved : " + username);
    }
}
```

src/test/java/testcases/LoginTest.java

LoginTest.java — src/test/java/testcases/LoginTest.java

```
package testcases;

import org.testng.Assert;
import org.testng.annotations.Test;

import com.parabank.base.BaseTest;

import pages.LoginPage;
import utilities.ExcelUtils;

public class LoginTest extends BaseTest {

    @Test(priority = 2)
    public void verifyValidLogin() {

        String username =
            ExcelUtils.getCellData("LoginData", 1, 0).trim();

        String password =
            ExcelUtils.getCellData("LoginData", 1, 1).trim();

        System.out.println("Username Read From Excel: " + username);
        System.out.println("Password Read From Excel: " + password);

        Assert.assertFalse(
            username.isEmpty(),
            "Username is empty in LoginData sheet");

        Assert.assertFalse(
            password.isEmpty(),
            "Password is empty in LoginData sheet");
    }
}
```

LoginTest.java — src/test/java/testcases/LoginTest.java

```
LoginPage loginPage = new LoginPage(driver);

loginPage.login(username, password);

Assert.assertTrue(
    loginPage.isLoginSuccessful(),
    "Valid login failed. Application message: "
    + loginPage.getLoginErrorMessage());

System.out.println("✓ Login Successful");
}
```

[src/test/java/testcases/AccountOverviewTest.java](#)

AccountOverviewTest.java — src/test/java/testcases/AccountOverviewTest.java

```
package testcases;

import org.testng.Assert;
import org.testng.annotations.Test;

import com.parabank.base.BaseTest;

import pages.LoginPage;
import utilities.ExcelUtils;

public class AccountOverviewTest extends BaseTest {

    @Test(priority = 7)
    public void verifyAccountOverview() {

        String username = ExcelUtils.getCellData("LoginData", 1, 0);

        String password = ExcelUtils.getCellData("LoginData", 1, 1);

        LoginPage loginPage = new LoginPage(driver);

        loginPage.login(username, password);

        Assert.assertTrue(driver.getPageSource().contains("Accounts Overview"));
    }
}
```

src/test/java/testcases/OpenAccountTest.java

OpenAccountTest.java — src/test/java/testcases/OpenAccountTest.java

```
package testcases;

import org.testng.Assert;
import org.testng.annotations.Test;

import com.parabank.base.BaseTest;

import pages.LoginPage;
import pages.OpenAccountPage;
import utilities.ExcelUtils;

public class OpenAccountTest extends BaseTest {

    @Test(priority = 4)
    public void verifyOpenAccount() {

        String username = ExcelUtils.getCellData("LoginData", 1, 0);

        String password = ExcelUtils.getCellData("LoginData", 1, 1);

        LoginPage loginPage = new LoginPage(driver);

        loginPage.login(username, password);

        OpenAccountPage openAccountPage = new OpenAccountPage(driver);

        String accountNumber = openAccountPage.openSavingsAccount();

        Assert.assertFalse(accountNumber.isEmpty());
        System.out.println("✓ Account Created Successfully");
        Assert.assertFalse(accountNumber.trim().isEmpty(), "Account number not generated");
    }
}
```

src/test/java/testcases/TransferFundsTest.java

TransferFundsTest.java — src/test/java/testcases/TransferFundsTest.java

```
package testcases;

import org.testng.Assert;
import org.testng.annotations.Test;

import com.parabank.base.BaseTest;
```

```
import pages.AccountOverviewPage;
import pages.LoginPage;
import pages.TransferFundsPage;
import utilities.ExcelUtils;

public class TransferFundsTest extends BaseTest {

    @Test(priority = 5)
    public void verifyTransferFunds() throws InterruptedException {

        String username = ExcelUtils.getCellData("LoginData", 1, 0);
        String password = ExcelUtils.getCellData("LoginData", 1, 1);

        LoginPage loginPage = new LoginPage(driver);
        loginPage.login(username, password);

        Assert.assertTrue(driver.getPageSource().contains("Accounts Overview"), "Login Failed");

        System.out.println("✓ Login Verified");

        AccountOverviewPage accountPage = new AccountOverviewPage(driver);

        double beforeBalance = accountPage.getCurrentBalance();

        System.out.println("Balance Before Transfer : " + beforeBalance);

        double transferAmount = 10;

        TransferFundsPage transferPage = new TransferFundsPage(driver);

        transferPage.validateTransferAmount(beforeBalance, transferAmount);

        String[] accounts = transferPage.transferFunds(String.valueOf(transferAmount));

        String fromAccount = accounts[0];
        String toAccount = accounts[1];

        System.out.println("✓ Money transferred from Account " + fromAccount + " to Account " + toAccount);

        Assert.assertTrue(driver.getPageSource().contains("Transfer Complete"), "Transfer Failed");
        System.out.println("✓ Transfer Successful");

        double expectedBalance = beforeBalance - transferAmount;

        Thread.sleep(2000);

        double afterBalance = accountPage.getCurrentBalance();

        System.out.println("Balance After Transfer : " + afterBalance);

        Assert.assertEquals(afterBalance, expectedBalance, 0.01, "Balance mismatch after transfer");
```

```
        System.out.println("✓ Remaining Balance Verified");
    }
}
```

src/test/java/testcases/BillPayTest.java

```
package testcases;

import org.openqa.selenium.By;
import org.testng.Assert;
import org.testng.annotations.Test;

import com.parabank.base.BaseTest;

import pages.BillPayPage;
import pages.LoginPage;
import utilities.DataProviders;
import utilities.ExcelUtils;

public class BillPayTest extends BaseTest {

    @Test(
        priority = 6, dataProvider = "billPayData", dataProviderClass = DataProviders.class)
    public void verifyBillPayment(

        String payee,
        String address,
        String city,
        String state,
        String zip,
        String phone,
        String accountNo,
        String amount) {

        String username = ExcelUtils.getCellData("LoginData", 1, 0);

        String password = ExcelUtils.getCellData("LoginData", 1, 1);

        LoginPage loginPage = new LoginPage(driver);

        loginPage.login(username, password);

        BillPayPage billPayPage = new BillPayPage(driver);

        billPayPage.payBill(
```

BillPayTest.java — src/test/java/testcases/BillPayTest.java

```
        payee,
        address,
        city,
        state,
        zip,
        phone,
        accountNo,
        amount);

Assert.assertTrue(billPayPage.getPaymentMessage().contains("Bill Payment"));

System.out.println("✓ Bill Payment Successful");
}}
```

src/test/java/testcases/NegativeTest.java

NegativeTest.java — src/test/java/testcases/NegativeTest.java

```
package testcases;

import org.testng.Assert;
import org.testng.annotations.Test;

import com.parabank.base.BaseTest;

import pages.LoginPage;

public class NegativeTest extends BaseTest {

    @Test(priority = 1)
    public void invalidLoginTest() {

        LoginPage loginPage = new LoginPage(driver);

        loginPage.login("wronguser123", "wrongpass123");

        String errorMessage = loginPage.getLoginErrorMessage();

        Assert.assertTrue(errorMessage.contains("could not be verified"), "Invalid login error message not displayed");

        System.out.println("✓ Invalid Login Validation Verified");
    }

    @Test(priority = 2)
    public void emptyLoginTest() {

        LoginPage loginPage = new LoginPage(driver);
```

NegativeTest.java — src/test/java/testcases/NegativeTest.java

```
loginPage.login("", "");

String errorMessage = loginPage.getLoginErrorMessage();

Assert.assertTrue(errorMessage.contains("Please enter a username and password"),
    "Empty login error message not displayed");

System.out.println("✓ Empty Login Validation Verified");
}
```

End of Source Code

The screenshot displays an IDE interface with a project explorer on the left and a console window on the right.

Project Explorer (Left):

- src/main/resources
- src/test/java
 - com.parabank.base
 - listeners
 - pages
 - AccountOverviewPage.java
 - BillPayPage.java
 - LoginPage.java
 - OpenAccountPage.java
 - RegistrationPage.java
 - TransferFundsPage.java
 - reports
 - testcases
 - AccountOverviewTest.java
 - BillPayTest.java
 - LoginTest.java
 - NegativeTest.java
 - OpenAccountTest.java
 - RegistrationTest.java
 - TransferFundsTest.java
 - utilities
- src/test/resources
- Maven Dependencies
- JRE System Library [JavaSE-1.8]
- screenshots
- src
- target
- test-output
 - DockerFile
 - JenkinsFile
 - pom.xml
 - testng.xml
 - testng-negative.xml

Code Editor (Top Right):

```

19 String password =
20     ExcelUtils.getCellData("LoginData", 1, 1).trim();
21
22 System.out.println("Username Read From Excel: " + username);
23 System.out.println("Password Read From Excel: " + password);
  
```

Console Window (Bottom Right):

Problems @ Javadoc Declaration Console X Results of running suite Coverage

<terminated> ParaBankAutomationFramework_testng.xml [TestNG] C:\Program Files\Java\jdk-17\bin\javaw.exe (12-Jun-2026, 9:39:48 pm – 9:40:38 pm elapsed 0:00:49) [pid: 1936]

[RemoteTestNG] detected TestNG version 7.11.0

- ✓ Registration Successful
- Username Saved : suraj1781280594311
- ✓ verifyRegistration PASSED
- Username Read From Excel: suraj1781280594311
- Password Read From Excel: Test@123
- ✓ Login Successful
- ✓ verifyValidLogin PASSED
- Account Number = 18228
- ✓ Account Created Successfully
- ✓ verifyOpenAccount PASSED
- ✓ Login Verified
- Balance Before Transfer : 415.5
- From Account : 17451
- To Account : 18228
- ✓ Money transferred from Account 17451 to Account 18228
- ✓ Transfer Successful
- Balance After Transfer : 405.5
- ✓ Remaining Balance Verified
- ✓ verifyTransferFunds PASSED
- ✓ Bill Payment Successful
- ✓ verifyBillPayment PASSED
- ✓ Bill Payment Successful
- ✓ verifyBillPayment PASSED
- ✓ Invalid Login Validation Verified
- ✓ invalidLoginTest PASSED
- ✓ Empty Login Validation Verified
- ✓ emptyLoginTest PASSED

=====

ParaBank Suite

Total tests run: 8, Passes: 8, Failures: 0, Skips: 0

=====

#23 - milestone project - Jenkin

Ask Gemini

localhost:8080/job/milestone%20project/23/stages/

Jenkins / milestone project / #23 / Stages

Started by Suraj Shaw · Started 2 min 37 sec ago · Queued 7 ms · Took 1 min 21 sec · Changes Tests Artifacts

Start

End

Tool Install

Checkout

Verify Tools

Prepare Selenium Grid

Build Docker Image

Run Tests in Docker

Post Actions

Search

Tool Install 0.25s

Checkout 3s

Verify Tools 1s

Prepare Selenium Grid 6s

Build Docker Image 3s

Run Tests in Docker 59s

Post Actions 1s

Post Actions 1s Started 1m 17s ago Jenkins

Archive JUnit-formatted test results 64ms

Archive the artifacts 83ms

Publish HTML reports 44ms

Checks if running on a Unix-like node 35ms

Windows Batch Script docker rm -f %TEST_CONTAINER% docker rm -f %SELENIUM_CONTAINER% docker image rm -f %TEST_IMAGE% 0.85s

ParaBank Docker test execution completed successfully.

08:35:34 ParaBank Docker test execution completed successfully.

Jenkins / milestone project

StatusChangesBuild NowConfigureDelete PipelineParaBank Extent ReportStagesRenamePipeline Syntax

milestone project

Last Successful ArtifactsLatest Test Result (no failures)

Permalinks

- Last build (#23), 23 min ago
- Last stable build (#23), 23 min ago
- Last successful build (#23), 23 min ago
- Last failed build (#21), 36 min ago
- Last unsuccessful build (#21), 36 min ago
- Last completed build (#23), 23 min ago

Add description

Test Result Trend

PassedSkippedFailed

Build	Passed	Skipped	Failed
#4	5	2	3
#6	8	2	2
#8	5	2	7
#10	5	2	7
#12	5	2	7
#14	5	2	7
#16	8	2	4
#18	5	2	7
#20	5	2	7
#23	5	2	17

```
Windows PowerShell
INFO: Using OpenTelemetry for tracing
✓ Registration Successful
Username Saved : suraj1781404505545
✓ verifyRegistration PASSED
Username Read From Excel: suraj1781404505545
Password Read From Excel: Test@123
✓ Login Successful
✓ verifyValidLogin PASSED
Account Number = 13899
✓ Account Created Successfully
✓ verifyOpenAccount PASSED
✓ Login Verified
Balance Before Transfer : 415.5
From Account : 13788
To Account : 13899
✓ Money transferred from Account 13788 to Account 13899
✓ Transfer Successful
Balance After Transfer : 405.5
✓ Remaining Balance Verified
✓ verifyTransferFunds PASSED
✓ Bill Payment Successful
✓ verifyBillPayment PASSED
✓ Bill Payment Successful
✓ verifyBillPayment PASSED
✓ Invalid Login Validation Verified
✓ invalidLoginTest PASSED
✓ Empty Login Validation Verified
✓ emptyLoginTest PASSED
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 50.15 s -- in TestSuite
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 53.634 s
[INFO] Finished at: 2026-06-14T02:35:48Z
[INFO] -----
PS D:\ParaBankAutomationFramework\ParaBankAutomationFramework\ParaBankAutomationFramework> |
```